

소개

동시성 제어와 데이터 정합성을 직접 재현하고 측정해 온 신입 백엔드 개발자입니다.

좌석 예약 시스템에서 락 전략 3종을 같은 조건으로 실측 비교해 중복 판매 0건을 검증했고, 혼합 부하에서 Redis 재고 선차감으로 쓰기 p95를 37ms에서 6ms로 줄였습니다.

Testcontainers 통합 테스트와 k6 부하 테스트로 주장에 근거를 붙이고, 측정하지 못한 항목은 측정하지 못했다고 문서에 남기는 방식으로 일합니다.

프로젝트

좌석 예약 시스템 — 대기열·락 전략·이벤트 전달

2026.02 — 2026.05 · 개인 프로젝트

락 전략 3종을 같은 조건에서 실측 비교하고 이벤트 복구 경로까지 검증한 예약 백엔드

Java 21, Spring Boot 3.4.1, PostgreSQL 16, Redis 7(Redisson 3.40.2), Apache Kafka(cp 7.6.0), JPA, Flyway, Testcontainers, k6 v1.5.0

- 동일 좌석 100명 동시 예매 경합에서 비관·낙관·Redis 분산 락 3전략을 같은 조건으로 비교해 중복 판매 0건 검증, p95 106-215ms 실측
- 서로 다른 좌석인데도 낙관적 락 성공률이 40%로 하락한 원인을 잔여석 공유 row의 버전 충돌로 규명하고 retry 한도 의존성까지 문서화
- 혼합 부하에서 소진된 좌석 요청이 DB 커넥션을 잡지 않도록 Redis 재고를 선차감해 쓰기 p95 37ms → 6ms, 총 RPS 969 → 1,005
- 결제·만료 race, 먹등 replay, 대기열 토큰 우회를 k6 시나리오로 재현해 3전략 × 3회 반복 체크 594/594 통과, 중복 결제 0건
- Transactional Outbox로 커밋과 발행을 분리하고 Kafka DLT 수동 replay 복구 경로를 Testcontainers 통합 테스트로 고정

DB 커밋 이후에만 브로드캐스트하는 메시지 파이프라인과 쿼리·인덱스 최적화

Java 21, Spring Boot 3.4.3, WebSocket(STOMP), Apache Kafka 3.9.0(KRaft), Redis 7 Pub/Sub, PostgreSQL 16, JPA, Testcontainers, k6 v1.5.0

- 채팅방 목록 조회의 2N+1 쿼리를 JPQL 프로젝션으로 제거하고 이후 추가 기능도 IN 배치로 처리해 방 50개 기준 101회 → 3회 고정
- 커서 페이지네이션·먹등성 체크·unread 계산 등 핵심 쿼리 4개를 EXPLAIN ANALYZE로 분석해 인덱스 5개 설계, 커버되는 3개는 근거를 적고 미추가
- DB 커밋 이후에만 브로드캐스트하도록 순서를 강제하고 발신자 ID + 클라이언트 메시지 ID 유니크 제약으로 재전송 먹등성 확보
- Redis 패턴 구독에서 수신 채널명을 전송 목적지로 쓰던 오브로드캐스트를 payload 기준으로 수정하고 단위 테스트로 고정
- Redis Cache Aside(TTL 5분)에서 잦은 이벤트만 선택 무효화하고 커밋 이후에 실행 — 메시지 수신은 방 멤버 키만, 읽음 처리는 해당 사용자 키만 제거

사용량 과금 게이트웨이 — 멀티테넌트 계량·정산

2026.05 · 개인 프로젝트

API Key 인증부터 사용량 계량, 정산 원장까지 돈이 걸린 경계의 정합성을 검증

Java 21, Spring Boot 3.5.14, Spring Security, PostgreSQL 16, Redis 7, JPA, Flyway, Testcontainers, k6

- 클라이언트 재시도가 중복 과금이 되지 않도록 사용량 기록에 Idempotency-Key를 강제해 중복 계량 0건 확인
- PG webhook 재전달을 HMAC 서명 검증과 이벤트 ID 중복 제거로 걸러 중복 결제 반영 0건 확인
- 게이트웨이·계량·인보이스·webhook 4개 경로를 모두 실행하는 혼합 시나리오 3회 반복에서 체크 150/150 통과, HTTP 실패 0건
- 잔액을 덮어쓰지 않는 append-only 원장으로 환불·조정을 포함한 금액 변화 이력을 추적 가능하게 설계
- API Key 해시 저장과 조직 스코프 격리로 멀티테넌트 교차 접근 차단

배리어프리 길찾기 My ETA — 교통약자 경로 안내

2026 (하나금융×SKT Tech4Good 해커톤) · 개발 4명(본인 BE 전담) / 기획·리서치 3명

교통약자의 실제 보행속도로 도착시간을 다시 계산하는 개인화 경로 API

Python 3.12, FastAPI, Pydantic, HTTPX, TMAP·서울 공공데이터·Kakao Local API, pytest, OpenAPI 3.1

- 표준 보행속도 기준 ETA가 교통약자에게 항상 틀리는 문제를 이동 유형·보조기구 프로필과 실측 속도 표본 기반 보정 엔진으로 해결
- 확인되지 않은 접근성 정보를 이용 가능으로 단정하지 않도록 UNKNOWN을 API 계약의 일급 상태로 정의
- 경로 이탈·대중교통 놓침을 감지해 현재 위치를 새 출발점으로 경로와 ETA를 재계산하는 흐름 구현
- TMAP 경로·서울 버스/지하철 실시간·엘리베이터 등 외부 API 5종을 provider 어댑터 계층으로 통합
- 유효하지 않은 속도 표본을 걸러 개인화 ETA에 반영되지 않게 하고, 원본 위치 좌표는 안내 중 계산에만 사용해 영구 저장하지 않도록 설계

활동

